

1. Introducción

Este documento reúne el diseño conceptual y la implementación de la base de datos relacional AlkeWallet, desarrollada para gestionar usuarios, monedas y transacciones de un monedero virtual. Se incluyen el modelo entidad-relación, el script de definición de datos (DDL), las sentencias de manipulación de datos (DML), las consultas SQL solicitadas y las validaciones de integridad y transaccionalidad (ACID).

1.1 ¿Qué es una base de datos relacional?

Una base de datos relacional organiza la información en tablas (relaciones) compuestas por filas y columnas, donde cada fila representa un registro y cada columna un atributo. Las tablas se vinculan entre sí mediante claves primarias y foráneas, lo que permite representar relaciones entre entidades sin duplicar información.

Ventajas frente a otros modelos (por ejemplo, NoSQL):

- Integridad referencial: las claves foráneas garantizan que no existan registros huérfanos (por ejemplo, una transacción sin usuario asociado).
- Consistencia mediante restricciones: NOT NULL, UNIQUE, CHECK y PRIMARY KEY evitan datos inválidos o duplicados.
- Lenguaje estándar (SQL): permite consultas complejas (JOIN, agregaciones, subconsultas) de forma declarativa.
- Transacciones ACID: aseguran que operaciones críticas, como transferencias de dinero, se completen de forma íntegra o se reviertan por completo.
- Normalización: reduce la redundancia de datos y previene anomalías de actualización, inserción y borrado.

2. Modelo Entidad-Relación

El modelo identifica tres entidades fuertes: Usuario, Moneda y Transacción. Transacción depende existencialmente de Usuario y Moneda (no puede existir sin referenciarlos), por lo que sus claves foráneas son obligatorias (NOT NULL).

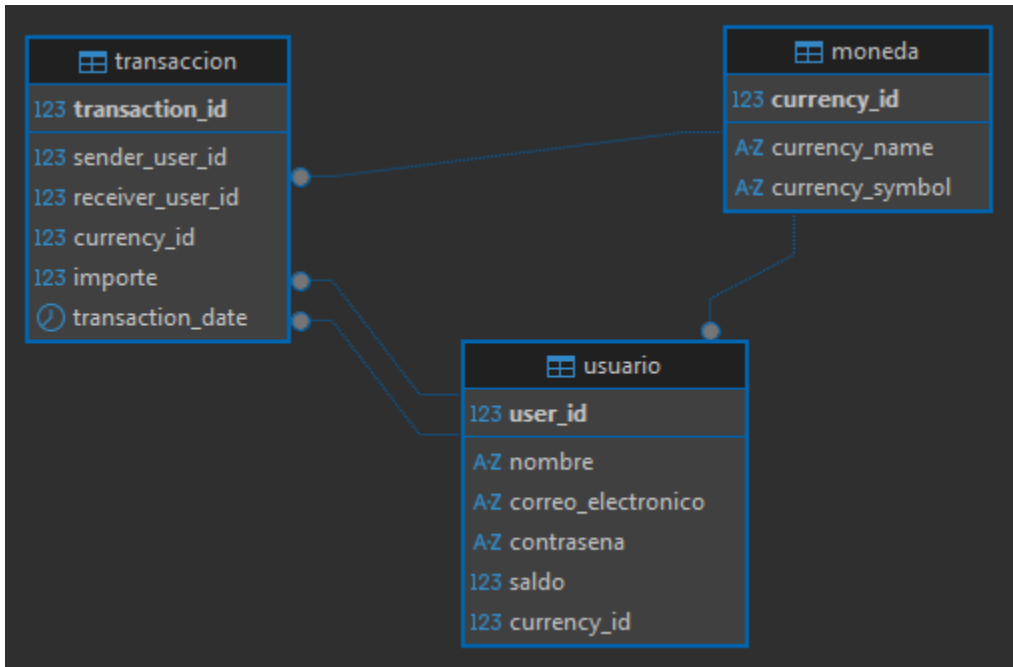


Figura 1. Diagrama Entidad-Relación de AlkeWallet.

2.1 Entidades y atributos

Entidad	Atributo	Tipo de dato sugerido	Restricción
usuario	user_id	INT AUTO_INCREMENT	PRIMARY KEY
usuario	nombre	VARCHAR(100)	NOT NULL
usuario	correo_electronico	VARCHAR(150)	NOT NULL, UNIQUE
usuario	contrasena	VARCHAR(255)	NOT NULL
usuario	saldo	DECIMAL(15,2)	NOT NULL, DEFAULT 0.00
usuario	currency_id	INT	FOREIGN KEY → moneda
usuario	fecha_creacion	DATETIME	DEFAULT CURRENT_TIMESTAMP (Tarea Plus)
moneda	currency_id	INT AUTO_INCREMENT	PRIMARY KEY
moneda	currency_name	VARCHAR(50)	NOT NULL
moneda	currency_symbol	VARCHAR(5)	NOT NULL
transaccion	transaction_id	INT AUTO_INCREMENT	PRIMARY KEY

transaccion	sender_user_id	INT	FOREIGN KEY → usuario, NOT NULL
transaccion	receiver_user_id	INT	FOREIGN KEY → usuario, NOT NULL
transaccion	currency_id	INT	FOREIGN KEY → moneda, NOT NULL
transaccion	importe	DECIMAL(15,2)	NOT NULL, CHECK (importe > 0)
transaccion	transaction_date	DATETIME	DEFAULT CURRENT_TIMESTAMP

2.2 Cardinalidades

Relación	Cardinalidad	Descripción
usuario — moneda (preferida)	N : 1	Un usuario elige una moneda; una moneda puede ser preferida por muchos usuarios.
usuario — transaccion (envía)	1 : N	Un usuario puede enviar muchas transacciones; cada transacción tiene un único emisor.
usuario — transaccion (recibe)	1 : N	Un usuario puede recibir muchas transacciones; cada transacción tiene un único receptor.
moneda — transaccion	1 : N	Una moneda puede estar asociada a muchas transacciones; cada transacción usa una única moneda.

2.3 Normalización (hasta 3FN)

- 1FN: todos los atributos son atómicos (por ejemplo, nombre y correo no se combinan en un solo campo) y no existen grupos repetitivos.
- 2FN: no hay dependencias parciales, ya que todas las tablas usan una clave primaria simple (sin claves compuestas).
- 3FN: no existen dependencias transitivas; por ejemplo, el nombre de la moneda (currency_name) no se almacena en usuario ni en transaccion, sino que se obtiene mediante JOIN con moneda a través de currency_id.

3. Script DDL — Creación de la base de datos

A continuación, el script completo para crear la base de datos AlkeWallet y sus tablas, ejecutado y verificado en SQLOnline (modo MySQL 8).

```

-- 1. Creación y selección de la base de datos
CREATE DATABASE AlkeWallet;
USE AlkeWallet;

-- 2. Tabla MONEDA
CREATE TABLE moneda (
  currency_id      INT AUTO_INCREMENT PRIMARY KEY,
  currency_name    VARCHAR(50) NOT NULL,
  currency_symbol  VARCHAR(5) NOT NULL
);

-- 3. Tabla USUARIO
CREATE TABLE usuario (
  user_id          INT AUTO_INCREMENT PRIMARY KEY,
  nombre           VARCHAR(100) NOT NULL,
  correo_electronico VARCHAR(150) NOT NULL UNIQUE,
  contrasena       VARCHAR(255) NOT NULL,
  saldo            DECIMAL(15,2) NOT NULL DEFAULT 0.00,
  currency_id      INT NOT NULL,
  CONSTRAINT fk_usuario_moneda
    FOREIGN KEY (currency_id) REFERENCES moneda(currency_id)
);

-- 4. Tabla TRANSACCION
CREATE TABLE transaccion (
  transaction_id    INT AUTO_INCREMENT PRIMARY KEY,
  sender_user_id    INT NOT NULL,
  receiver_user_id  INT NOT NULL,
  currency_id       INT NOT NULL,
  importe           DECIMAL(15,2) NOT NULL CHECK (importe > 0),
  transaction_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_transaccion_sender
    FOREIGN KEY (sender_user_id) REFERENCES usuario(user_id),
  CONSTRAINT fk_transaccion_receiver
    FOREIGN KEY (receiver_user_id) REFERENCES usuario(user_id),
  CONSTRAINT fk_transaccion_moneda
    FOREIGN KEY (currency_id) REFERENCES moneda(currency_id),
  INDEX idx_sender_fecha (sender_user_id, transaction_date),
  INDEX idx_receiver_fecha (receiver_user_id, transaction_date)
);

-- 5. Verificación
SHOW DATABASES;
SHOW TABLES;
DESCRIBE usuario;
DESCRIBE moneda;
DESCRIBE transaccion;

```

3.1 Tarea Plus — ALTER TABLE

Se agregó posteriormente la columna fecha_creacion a la tabla usuario:

```

ALTER TABLE usuario
  ADD COLUMN fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP;

```

```
ALTER TABLE usuario ADD COLUMN fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP;

-- 4. Sentencias DML — Carga y manipulación de datos
-- 1. Inserción de datos de prueba
-- Monedas
INSERT INTO moneda (currency_name, currency_symbol) VALUES
    ('Peso Chileno', 'CLP'),
    ('Dólar Estadounidense', 'USD'),
    ('Euro', 'EUR');

-- Usuarios
```

Name	Value
Updated Rows	0
Execute time	0,058s
Start time	Thu Jul 02 19:30:45 CLT 2026
Finish time	Thu Jul 02 19:30:45 CLT 2026
Query	ALTER TABLE usuario ADD COLUMN fecha_creacion DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP

4. Sentencias DML — Carga y manipulación de datos

4.1 Inserción de datos de prueba

```
-- Monedas
INSERT INTO moneda (currency_name, currency_symbol) VALUES
    ('Peso Chileno', 'CLP'),
    ('Dólar Estadounidense', 'USD'),
    ('Euro', 'EUR');

-- Usuarios
INSERT INTO usuario (nombre, correo_electronico, contrasena, saldo, currency_id) VALUES
    ('Ana Torres', 'ana.torres@mail.com', 'hash_pw_1', 150000.00, 1),
    ('Bruno Salas', 'bruno.salas@mail.com', 'hash_pw_2', 50000.00, 1),
    ('Carla Fuentes', 'carla.fuentes@mail.com', 'hash_pw_3', 300.00, 2);

-- Transacciones
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe) VALUES
    (1, 2, 1, 20000.00),
    (2, 3, 1, 5000.00),
    (3, 1, 2, 15.50);
```

```
INSERT INTO moneda (currency_name, currency_symbol) VALUES
('Peso Chileno', 'CLP'),
('Dólar Estadounidense', 'USD'),
('Euro', 'EUR');

-- Usuarios

INSERT INTO usuario (nombre, correo_electronico, contrasena, saldo, currency_id) VALUES
('Ana Torres', 'ana.torres@mail.com', 'hash_pw_1', 150000.00, 1),
('Bruno Salas', 'bruno.salas@mail.com', 'hash_pw_2', 50000.00, 1),
('Carla Fuentes', 'carla.fuentes@mail.com', 'hash_pw_3', 300.00, 2);

-- Transacciones

INSERT INTO transaction (sender_user_id, receiver_user_id, currency_id, importe) VALUES
(1, 2, 1, 20000.00),
```

Estadísticas 1	
Name	Value
Updated Rows	3
Execute time	0,005s
Start time	Thu Jul 02 19:30:57 CLT 2026
Finish time	Thu Jul 02 19:30:57 CLT 2026
Query	INSERT INTO moneda (currency_name, currency_symbol) VALUES ('Peso Chileno', 'CLP'), ('Dólar Estadounidense', 'USD'), ('Euro', 'EUR')

```
INSERT INTO usuario (nombre, correo_electronico, contrasena, saldo, currency_id) VALUES
('Ana Torres', 'ana.torres@mail.com', 'hash_pw_1', 150000.00, 1),
('Bruno Salas', 'bruno.salas@mail.com', 'hash_pw_2', 50000.00, 1),
('Carla Fuentes', 'carla.fuentes@mail.com', 'hash_pw_3', 300.00, 2);

-- Transacciones

INSERT INTO transaction (sender_user_id, receiver_user_id, currency_id, importe) VALUES
(1, 2, 1, 20000.00),
(2, 3, 1, 5000.00),
(3, 1, 2, 15.50);
```

Estadísticas 1	
Name	Value
Updated Rows	3
Execute time	0,004s
Start time	Thu Jul 02 19:31:08 CLT 2026
Finish time	Thu Jul 02 19:31:08 CLT 2026
Query	INSERT INTO usuario (nombre, correo_electronico, contrasena, saldo, currency_id) VALUES ('Ana Torres', 'ana.torres@mail.com', 'hash_pw_1', 150000.00, 1), ('Bruno Salas', 'bruno.salas@mail.com', 'hash_pw_2', 50000.00, 1), ('Carla Fuentes', 'carla.fuentes@mail.com', 'hash_pw_3', 300.00, 2)

The screenshot shows a database IDE with a SQL editor and a statistics window. The SQL editor contains the following code:

```
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe) VALUES
(1, 2, 1, 20000.00),
(2, 3, 1, 5000.00),
(3, 1, 2, 15.50);

-- 2. Transacción controlada (ACID) - actualización de saldo

START TRANSACTION;

-- Se descuenta el importe al emisor

UPDATE usuario
SET saldo = saldo - 20000.00
```

The statistics window, titled "Estadísticas 1", shows the following data:

Name	Value
Updated Rows	3
Execute time	0,005s
Start time	Thu Jul 02 19:31:19 CLT 2026
Finish time	Thu Jul 02 19:31:19 CLT 2026
Query	INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe) VALUES (1, 2, 1, 20000.00), (2, 3, 1, 5000.00), (3, 1, 2, 15.50)

4.2 Transacción controlada (ACID) — actualización de saldo

Al registrar una transferencia, el saldo del emisor y del receptor deben actualizarse dentro de la misma transacción de base de datos, de modo que ambas operaciones se confirmen juntas (COMMIT) o se reviertan juntas (ROLLBACK) si ocurre un error.

```
START TRANSACTION;

-- Se descuenta el importe al emisor
UPDATE usuario
  SET saldo = saldo - 20000.00
 WHERE user_id = 1;

-- Se acredita el importe al receptor
UPDATE usuario
  SET saldo = saldo + 20000.00
 WHERE user_id = 2;

-- Se registra la transacción
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
  VALUES (1, 2, 1, 20000.00);

-- Si todo fue correcto:
COMMIT;
```

-- 2. Transacción controlada (ACID) - actualización de saldo

START TRANSACTION;

-- Se descuenta el importe al emisor

UPDATE usuario

SET saldo = saldo - 20000.00

WHERE user_id = 1;

-- Se acredita el importe al receptor

UPDATE usuario

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0s
Start time	Thu Jul 02 19:31:35 CLT 2026
Finish time	Thu Jul 02 19:31:35 CLT 2026
Query	START TRANSACTION

-- Se descuenta el importe al emisor

UPDATE usuario

SET saldo = saldo - 20000.00

WHERE user_id = 1;

-- Se acredita el importe al receptor

UPDATE usuario

SET saldo = saldo + 20000.00

WHERE user_id = 2;

Estadísticas 1

Name	Value
Updated Rows	1
Execute time	0,003s
Start time	Thu Jul 02 19:31:44 CLT 2026
Finish time	Thu Jul 02 19:31:44 CLT 2026
Query	UPDATE usuario SET saldo = saldo - 20000.00 WHERE user_id = 1


```
-- Se acredita el importe al receptor

UPDATE usuario

SET saldo = saldo + 20000.00

WHERE user_id = 2;

-- Se registra la transacción

INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)

VALUES (1, 2, 1, 20000.00);

-- Si todo fue correcto:
```

Estadísticas 1

Name	Value
Updated Rows	1
Execute time	0,003s
Start time	Thu Jul 02 19:31:54 CLT 2026
Finish time	Thu Jul 02 19:31:54 CLT 2026
Query	UPDATE usuario SET saldo = saldo + 20000.00 WHERE user_id = 2

```
-- Se registra la transacción

INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)

VALUES (1, 2, 1, 20000.00);

-- Si todo fue correcto:

COMMIT;

-- 3. Simulación de error de integridad referencial y ROLLBACK

START TRANSACTION;
```

Estadísticas 1

Name	Value
Updated Rows	1
Execute time	0,001s
Start time	Thu Jul 02 19:32:03 CLT 2026
Finish time	Thu Jul 02 19:32:03 CLT 2026
Query	INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe) VALUES (1, 2, 1, 20000.00)

```
-- Si todo fue correcto:
COMMIT;

-- 3. Simulación de error de integridad referencial y ROLLBACK
START TRANSACTION;

UPDATE usuario SET saldo = saldo - 999999999.00 WHERE user_id = 1;

-- Se intenta insertar una transacción con un usuario receptor inexistente (viola la FK)
```

Name	Value
Updated Rows	0
Execute time	0,002s
Start time	Thu Jul 02 19:32:12 CLT 2026
Finish time	Thu Jul 02 19:32:12 CLT 2026
Query	COMMIT

4.3 Simulación de error de integridad referencial y ROLLBACK

```
START TRANSACTION;

UPDATE usuario SET saldo = saldo - 999999999.00 WHERE user_id = 1;

-- Se intenta insertar una transacción con un usuario receptor inexistente (viola la FK)
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
VALUES (1, 999, 1, 999999999.00);
-- ERROR 1452: Cannot add or update a child row: a foreign key constraint fails

-- Ante el error, se revierten todos los cambios de la transacción
ROLLBACK;
```

```
-- 3. Simulación de error de integridad referencial y ROLLBACK
START TRANSACTION;

UPDATE usuario SET saldo = saldo - 999999999.00 WHERE user_id = 1;

-- Se intenta insertar una transacción con un usuario receptor inexistente (viola la FK)
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
VALUES (1, 999, 1, 999999999.00);

-- ERROR 1452: Cannot add or update a child row: a foreign key constraint fails

-- Ante el error, se revierten todos los cambios de la transacción
ROLLBACK;
```

Name	Value
Updated Rows	0
Execute time	0,001s
Start time	Thu Jul 02 19:32:23 CLT 2026
Finish time	Thu Jul 02 19:32:23 CLT 2026
Query	START TRANSACTION

```
-- 3. Simulación de error de integridad referencial y ROLLBACK

START TRANSACTION;

UPDATE usuario SET saldo = saldo - 999999999.00 WHERE user_id = 1;

-- Se intenta insertar una transacción con un usuario receptor inexistente (viola la FK)
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
VALUES (1, 999, 1, 999999999.00);

-- ERROR 1452: Cannot add or update a child row: a foreign key constraint fails
-- Ante el error, se revierten todos los cambios de la transacción

ROLLBACK;
```

Name	Value
Updated Rows	1
Execute time	0,001s
Start time	Thu Jul 02 19:32:31 CLT 2026
Finish time	Thu Jul 02 19:32:31 CLT 2026
Query	UPDATE usuario SET saldo = saldo - 999999999.00 WHERE user_id = 1

```
-- Se intenta insertar una transacción con un usuario receptor inexistente (viola la FK)
INSERT INTO transaccion (sender_user_id, receiver_user_id, currency_id, importe)
VALUES (1, 999, 1, 999999999.00);

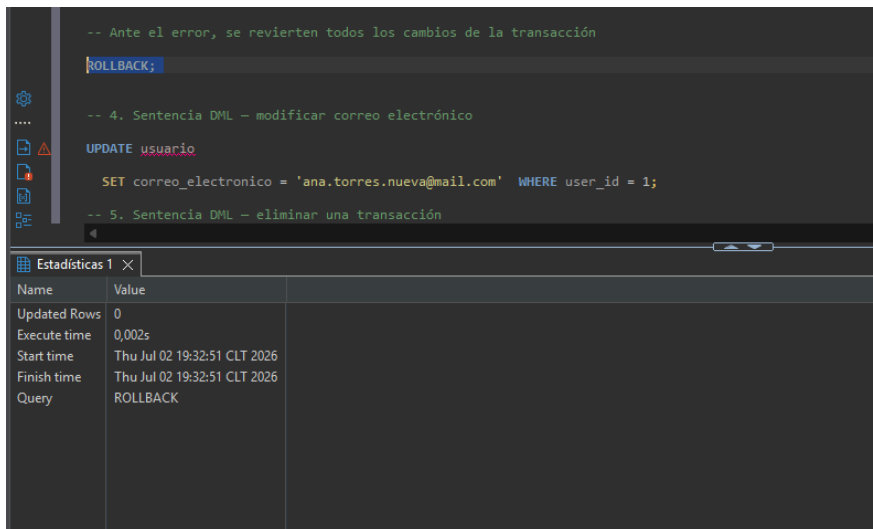
-- ERROR 1452: Cannot add or update a child row: a foreign key constraint fails
-- Ante el error, se revierten todos los cambios de la transacción

ROLLBACK;
```

```
-- 4. Sentencia DML - modificar correo electrónico
```

Name	Value
Updated Rows	1
Execute time	0,001s
Start time	Thu Jul 02 19:32:31 CLT 2026
Finish time	Thu Jul 02 19:32:31 CLT 2026
Query	UPDATE usuario SET correo = 'nuevo@ejemplo.com' WHERE user_id = 1

SQL Error [1452] [23000]: Cannot add or update a child row: a foreign key constraint fails ('alkewallet`.`transaccion', CONSTRAINT 'fk_transaccion_receiver' FOREIGN KEY ('receiver_user_id') REFERENCES 'usuario' ('user_id'))



4.4 Propiedades ACID aplicadas

Propiedad	Aplicación en AlkeWallet
Atomicidad	El descuento al emisor, el abono al receptor y el registro de la transacción se ejecutan como una sola unidad: o se aplican todos los cambios, o ninguno (COMMIT / ROLLBACK).
Consistencia	Las restricciones (FOREIGN KEY, NOT NULL, CHECK importe > 0) impiden que la base de datos quede en un estado inválido, como una transacción sin usuario asociado.
Aislamiento	Mientras una transacción de transferencia está en curso, otras conexiones no ven los saldos parcialmente actualizados hasta que se confirma el COMMIT.
Durabilidad	Una vez ejecutado el COMMIT, los cambios en saldo y transaccion persisten aunque ocurra una falla posterior del sistema.

4.5 Sentencia DML — modificar correo electrónico

```
UPDATE usuario
SET correo_electronico = 'ana.torres.nueva@mail.com'
WHERE user_id = 1;
```

```
-- 4. Sentencia DML - modificar correo electrónico
UPDATE usuario
SET correo_electronico = 'ana.torres.nueva@mail.com' WHERE user_id = 1;

-- 5. Sentencia DML - eliminar una transacción
DELETE FROM transaccion WHERE transaction_id = 3;

-- 5. Consultas SQL (SELECT)
```

Name	Value
Updated Rows	1
Execute time	0,004s
Start time	Thu Jul 02 19:33:02 CLT 2026
Finish time	Thu Jul 02 19:33:02 CLT 2026
Query	UPDATE usuario SET correo_electronico = 'ana.torres.nueva@mail.com' WHERE user_id = 1

4.6 Sentencia DML — eliminar una transacción

```
DELETE FROM transaccion
WHERE transaction_id = 3;
```

```
-- 5. Sentencia DML - eliminar una transacción
DELETE FROM transaccion WHERE transaction_id = 3;

-- 5. Consultas SQL (SELECT)

-- 1. Nombre de la moneda elegida por un usuario específico
SELECT u.user_id, u.nombre, m.currency_name, m.currency_symbol
```

Name	Value
Updated Rows	1
Execute time	0,003s
Start time	Thu Jul 02 19:33:14 CLT 2026
Finish time	Thu Jul 02 19:33:14 CLT 2026
Query	DELETE FROM transaccion WHERE transaction_id = 3

5. Consultas SQL (SELECT)

5.1 Nombre de la moneda elegida por un usuario específico

```
SELECT u.user_id, u.nombre, m.currency_name, m.currency_symbol
FROM usuario u
INNER JOIN moneda m ON u.currency_id = m.currency_id
WHERE u.user_id = 1;
```

```
-- 5. Consultas SQL (SELECT)

-- 1. Nombre de la moneda elegida por un usuario específico

SELECT u.user_id, u.nombre, m.currency_name, m.currency_symbol
FROM usuario u
INNER JOIN moneda m ON u.currency_id = m.currency_id
WHERE u.user_id = 1;

-- 2. Todas las transacciones registradas

SELECT transaction_id, sender_user_id, receiver_user_id,
```

usuario(+) 1 X

SELECT u.user_id, u.nombre, m.currency_name, m.currency_symbol Enter a SQL expression to filter results (use Ctrl+Space)

	123 user_id	AZ nombre	AZ currency_name	AZ currency_symbol
1	1	Ana Torres	Peso Chileno	CLP

5.2 Todas las transacciones registradas

```
SELECT transaction_id, sender_user_id, receiver_user_id,
       currency_id, importe, transaction_date
FROM transaccion
ORDER BY transaction_date DESC;
```

```
-- 2. Todas las transacciones registradas

SELECT transaction_id, sender_user_id, receiver_user_id,
       currency_id, importe, transaction_date
FROM transaccion
ORDER BY transaction_date DESC;

-- 3. Transacciones realizadas por un usuario específico

SELECT t.transaction_id, t.sender_user_id, t.receiver_user_id,
       t.importe, t.transaction_date,
```

transaccion 1 X

SELECT transaction_id, sender_user_id, receiver_user_id, currency_id, importe, transaction_date Enter a SQL expression to filter results (use Ctrl+Space)

	123 transaction_id	123 sender_user_id	123 receiver_user_id	123 currency_id	123 importe	transaction_date
1	4	1	2	1	20.000	2026-07-02 19:32:03
2	1	1	2	1	20.000	2026-07-02 19:31:19
3	2	2	3	1	5.000	2026-07-02 19:31:19

5.3 Transacciones realizadas por un usuario específico

```
SELECT t.transaction_id, t.sender_user_id, t.receiver_user_id,
       t.importe, t.transaction_date,
       CASE WHEN t.sender_user_id = 1 THEN 'Enviada' ELSE 'Recibida' END AS tipo
FROM transaccion t
WHERE t.sender_user_id = 1 OR t.receiver_user_id = 1
ORDER BY t.transaction_date DESC;
```

```
-- 3. Transacciones realizadas por un usuario específico
SELECT t.transaction_id, t.sender_user_id, t.receiver_user_id,
       t.importe, t.transaction_date,
       CASE WHEN t.sender_user_id = 1 THEN 'Enviada' ELSE 'Recibida' END AS tipo
FROM transaccion t
WHERE t.sender_user_id = 1 OR t.receiver_user_id = 1
ORDER BY t.transaction_date DESC;
```

● -- 4. Consultas adicionales (Lección 2 – JOIN, WHERE, agregaciones)
 -- SELECT básico con filtro dinámico:

```
SELECT nombre, correo_electronico, saldo
FROM usuario
WHERE saldo > 10000 AND currency_id = 1;
```

transaction 1 X

	123 transaction_id	123 sender_user_id	123 receiver_user_id	123 importe	transaction_date	AZ tipo
1	4	1	2	20.000	2026-07-02 19:32:03	Enviada
2	1	1	2	20.000	2026-07-02 19:31:19	Enviada

5.4 Consultas adicionales (Lección 2 — JOIN, WHERE, agregaciones)

SELECT básico con filtro dinámico:

```
SELECT nombre, correo_electronico, saldo
FROM usuario
WHERE saldo > 10000 AND currency_id = 1;
```

● -- 4. Consultas adicionales (Lección 2 – JOIN, WHERE, agregaciones)
 -- SELECT básico con filtro dinámico:

```
SELECT nombre, correo_electronico, saldo
FROM usuario
WHERE saldo > 10000 AND currency_id = 1;
```

-- JOIN entre transaccion y usuario (datos del emisor):

```
SELECT t.transaction_id, u.nombre AS emisor, t.importe, t.transaction_date
FROM transaccion t
INNER JOIN usuario u ON t.sender_user_id = u.user_id;
```

-- Subconsulta – total de transacciones enviadas por usuario:

```
SELECT u.user_id, u.nombre,
```

usuario 1 X

	AZ nombre	AZ correo_electronico	123 saldo
1	Ana Torres	ana.torres.nueva@mail.com	130.000
2	Bruno Salas	bruno.salas@mail.com	70.000

JOIN entre transaccion y usuario (datos del emisor):

```
SELECT t.transaction_id, u.nombre AS emisor, t.importe, t.transaction_date
FROM transaccion t
INNER JOIN usuario u ON t.sender_user_id = u.user_id;
```

```
-- JOIN entre transaccion y usuario (datos del emisor):
SELECT t.transaction_id, u.nombre AS emisor, t.importe, t.transaction_date
FROM transaccion t
INNER JOIN usuario u ON t.sender_user_id = u.user_id;

-- Subconsulta — total de transacciones enviadas por usuario:
SELECT u.user_id, u.nombre,
       (SELECT COUNT(*) FROM transaccion t WHERE t.sender_user_id = u.user_id) AS total_enviadas
FROM usuario u;

-- Tarea Plus — vista con el top-5 de usuarios con mayor saldo:
```

transaccion(+) 1				
	123 transaction_id	AZ emisor	123 importe	transaction_date
1	1	Ana Torres	20.000	2026-07-02 19:31:19
2	2	Bruno Salas	5.000	2026-07-02 19:31:19
3	4	Ana Torres	20.000	2026-07-02 19:32:03

Subconsulta — total de transacciones enviadas por usuario:

```
SELECT u.user_id, u.nombre,
       (SELECT COUNT(*) FROM transaccion t WHERE t.sender_user_id = u.user_id) AS total_enviadas
FROM usuario u;

-- Subconsulta — total de transacciones enviadas por usuario:
SELECT u.user_id, u.nombre,
       (SELECT COUNT(*) FROM transaccion t WHERE t.sender_user_id = u.user_id) AS total_enviadas
FROM usuario u;

-- Tarea Plus — vista con el top-5 de usuarios con mayor saldo:
CREATE VIEW vista_top5_saldo AS
SELECT user_id, nombre, saldo
FROM usuario
ORDER BY saldo DESC
```

usuario 1			
	123 user_id	AZ nombre	123 total_enviadas
1	1	Ana Torres	2
2	2	Bruno Salas	1
3	3	Carla Fuentes	0

Tarea Plus — vista con el top-5 de usuarios con mayor saldo:

```
CREATE VIEW vista_top5_saldo AS
SELECT user_id, nombre, saldo
FROM usuario
ORDER BY saldo DESC
LIMIT 5;

SELECT * FROM vista_top5_saldo;
```


-- Tarea Plus - vista con el top-5 de usuarios con mayor saldo:

```
CREATE VIEW vista_top5_saldo AS
SELECT user_id, nombre, saldo
FROM usuario
ORDER BY saldo DESC
LIMIT 5;
SELECT * FROM vista_top5_saldo;
```

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,005s
Start time	Thu Jul 02 19:34:34 CLT 2026
Finish time	Thu Jul 02 19:34:34 CLT 2026
Query	CREATE VIEW vista_top5_saldo AS SELECT user_id, nombre, saldo FROM usuario ORDER BY saldo DESC LIMIT 5

usuario 1

SELECT * FROM vista_top5_saldo

	123 user_id	AZ nombre	123 saldo
1	1	Ana Torres	130.000
2	2	Bruno Salas	70.000
3	3	Carla Fuentes	300

6. ¿Qué se validó?

6.1 Aspectos técnicos

- Diseño de la base de datos alineado al modelo entidad-relación (3 entidades, cardinalidades definidas).
- Integridad de los datos mediante NOT NULL, UNIQUE y CHECK (importe > 0).
- Uso de identificadores claros: user_id, currency_id, transaction_id como claves primarias autoincrementales.
- Integridad referencial garantizada con FOREIGN KEY entre usuario, moneda y transaccion.
- Uso completo de SQL: DDL (CREATE, ALTER), DML (INSERT, UPDATE, DELETE) y consultas (SELECT, JOIN, subconsultas, vistas).

6.2 Aspectos estructurales (ACID)

El diseño garantiza que las operaciones de transferencia de fondos sean atómicas, consistentes, aisladas y durables, tal como se demostró en la sección 4.2 y 4.3 mediante START TRANSACTION, COMMIT y ROLLBACK.